

PROVING EFFICIENCY

Time Complexity:

- Using Mathematical induction to prove that the time complexity of the algorithm is $O(n)$, where n is the number of cities.
- **Base Case ($n = 1$):**
 - For $n = 1$, the algorithm performs exactly one iteration of the loop. During this iteration, the operations such as calculating fuel balance and checking conditions take constant time, which is $O(1)$. Therefore, the time complexity for $n = 1$ is $O(1)$.
- **Inductive Hypothesis:**
 - Assume that for an input size k , the time complexity of the algorithm is $O(k)$. That is, if there are k cities, the algorithm iterates over all k cities, performing constant-time operations during each iteration, resulting in $O(k)$ time complexity.
- **Inductive Step ($n = k + 1$):**
 - Now, we consider the case where there are $k + 1$ cities. In this case, the algorithm performs exactly one additional iteration compared to the case with k cities. This additional iteration takes constant time $O(1)$, so the total time complexity for $k + 1$ cities is:
 - $T(k + 1) = T(k) + O(1)$
 - By the inductive hypothesis, we know that $T(k) = O(k)$, so:
 - $T(k + 1) = O(k) + O(1) = O(k + 1)$
- **Conclusion:**
 - By the principle of **mathematical induction**, the time complexity for n cities is $O(n)$, where n is the total number of cities. Therefore, the algorithm's time complexity grows linearly with the number of cities, confirming that the time complexity is $O(n)$.

Space Complexity:

- The algorithm only uses a constant amount of additional space, regardless of the size of the input:

1. Variables:

- The algorithm uses a few scalar variables (**total_fuel**, **current_fuel**, and **starting_city**), which require **O(1)** space.

2. Input Arrays:

- The input arrays **city_distances** and **fuel** are passed by reference, so no additional space is used to copy them.
- Therefore, the overall space complexity of the algorithm is:
 - **O(1)**, meaning it uses constant space.

CONCLUSION:

The algorithm is highly efficient, with a time complexity of **O(n)** and a space complexity of **O(1)**. It scales linearly with the number of cities, making it suitable for large inputs while requiring minimal memory.